

Student ID Login: Changes and Testing Guide

Branch: chore/development_server_new · Date: 9 October 2026 · Audience: backend/frontend developers and testers

Students who don't have an email address can now create an account with their **name and a password**. They get a generated **Student ID** (for example `ZV7K3M9Q`) and use it with their password to log in.

After login they receive the **same access and refresh tokens as Google login**. Courses, progress, quizzes, coding problems, submissions, the leaderboard, token refresh and logout all work for them exactly as for any other student.

Forgotten passwords: there is no "forgot password" flow. A student who forgets their password asks the team, and a **super admin** sets a new one with the admin API in this guide.

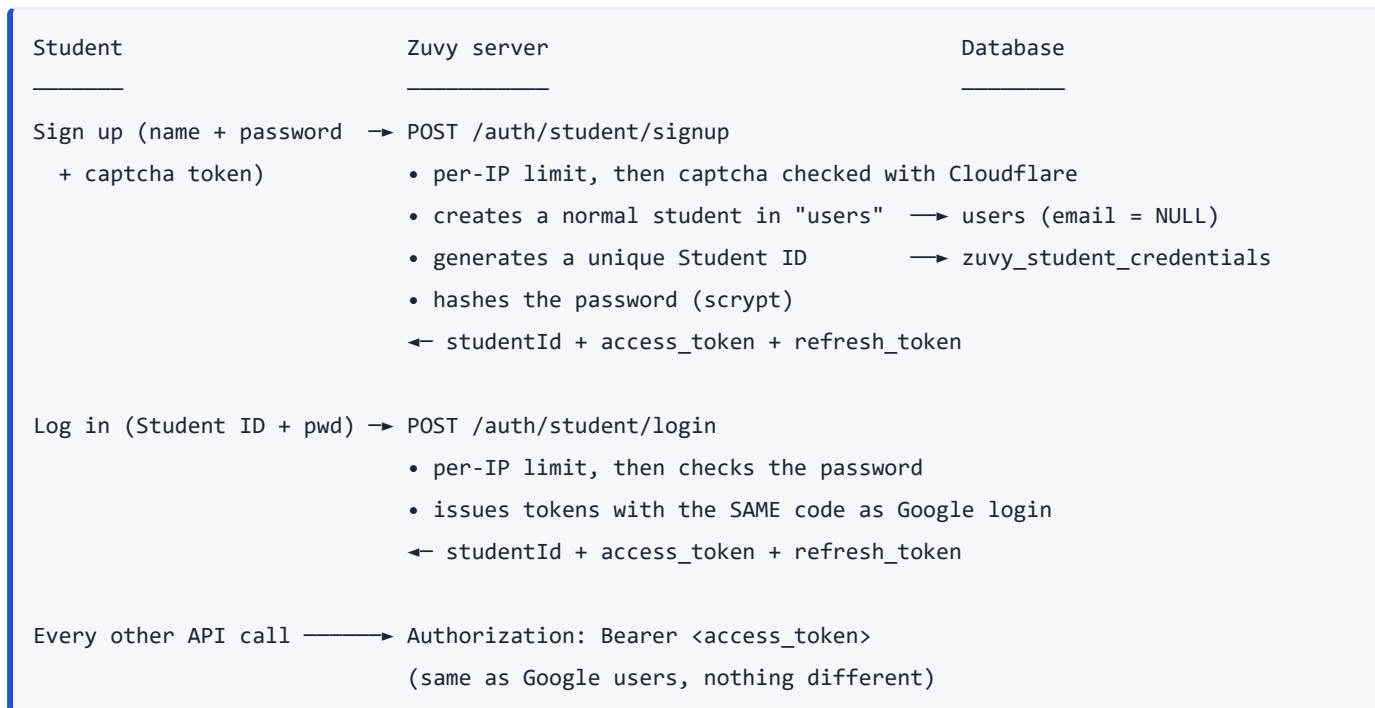
Abuse protection: every sign-up must pass a **Cloudflare Turnstile captcha, checked on the backend**. Both sign-up and login also have **per-IP limits**. The limits are high enough for a whole school lab sharing one Wi-Fi, and they use the student's real IP that nginx forwards (section 3).

Google login has not changed. Users with an email (admins, instructors and students) are not affected; section 2.3 lists every change made to existing code and why it is safe.

Contents

1. How it works
 2. What changed in the code
 3. Before you test: migration and server settings
 4. Using Swagger
 5. API reference with examples
 6. Test cases: step by step
 7. Regression tests for users with email
 8. Checking the database
 9. Known limitations
 10. Troubleshooting
 11. Notes for frontend developers
-

1. How it works



Key points:

- **Same user table:** the student is a normal row in `users`, with `email` and `google_user_id` both NULL and `mode = 'student'`.
- **Separate credentials table:** the Student ID and password hash live in the new table `zuvy_student_credentials`, which references `users.id` with a foreign key.
- **Student ID format:** `zv` plus 6 characters. Look-alike characters (0/O, 1/I/L) are never used, so the ID is easy to read and type. Login is **case-insensitive** (`zv7k3m9q` works too).
- **Password storage:** passwords are never stored in plain text, only as a salted `scrypt` hash.
- **Lockout:** after **5 wrong passwords**, the account is locked for **15 minutes**. The lock applies to that one account, so classmates on the same Wi-Fi are never affected.
- **Captcha:** sign-up needs a Turnstile token, verified by the backend with Cloudflare. Calling the API directly without a valid token fails, and each browser is judged on its own, not by IP.
- **Per-IP limits** (configurable, see 3.2):

Route	Default limit per IP
Sign-up	60 per minute and 300 per hour
Login	600 per 15 minutes

These are ceilings against floods, sized for a whole school lab behind one public IP. Other routes in the app have no rate limit.

- **Public courses:** the student is enrolled automatically when they open one (existing behaviour, unchanged).
- **Private courses:** a super admin enrolls the student by Student ID, using the API in section 5.5.

2. What changed in the code

2.1 New files

File	What it is
<code>drizzle/migrations/0043_create_zuvy_student_credentials.sql</code>	Creates the <code>zuvy_student_credentials</code> table
<code>src/student-auth/student-auth.module.ts</code>	New Nest module
<code>src/student-auth/student-auth.controller.ts</code>	The 5 new endpoints under <code>/auth/student</code>
<code>src/student-auth/student-auth.service.ts</code>	Sign-up, login, logout, password reset, search and enrol logic
<code>src/student-auth/dto/student-auth.dto.ts</code>	Request body and query validation
<code>src/student-auth/captcha.service.ts</code>	Verifies the Turnstile captcha token with Cloudflare on the backend
<code>src/student-auth/student-auth.throttle.ts</code>	Per-IP limits for sign-up and login (read from env)
<code>src/config/trust-proxy.ts</code>	Which proxies (nginx via Docker) are trusted for the real client IP
<code>src/student-auth/student-auth.service.spec.ts</code>	Unit tests for password hashing and Student ID generation
<code>src/student-auth/captcha.service.spec.ts</code>	Unit tests for the captcha check
<code>src/student-auth/student-auth.throttle.spec.ts</code>	Tests for the per-IP limits behind a proxy, including spoofed headers

2.2 New table: `zuvy_student_credentials`

Column	Type	Meaning
<code>id</code>	serial	Primary key
<code>user_id</code>	bigint, unique, FK → <code>users.id</code> (cascade delete)	The student
<code>student_id</code>	varchar(16), unique	Login ID, e.g. <code>ZV7K3M9Q</code>
<code>password_hash</code>	varchar(255)	<code>scrypt\$16384\$8\$1\$<salt>\$<hash></code>
<code>failed_attempts</code>	integer, default 0	Wrong passwords in a row
<code>locked_until</code>	timestampz, nullable	Login blocked until this time
<code>password_updated_at</code>	timestampz	Last password change
<code>last_reset_by</code>	bigint, FK → <code>users.id</code>	Super admin who last reset the password
<code>last_reset_at</code>	timestampz	When it was last reset
<code>created_at</code>	timestampz	When the account was created

2.3 Changes to existing files

Every change below is a **no-op for users who have an email**: they run exactly the same logic as before. Each change only affects rows where the email is NULL or empty.

File	Change	Why it was needed
<code>src/auth/auth.service.ts</code>	The token-issuing half of Google <code>login()</code> was moved into a new method, <code>issueLoginSession(user)</code> . Google login calls it unchanged.	Student ID login reuses exactly the same code, so both produce identical tokens.
<code>src/auth/auth.service.ts</code>	<code>isPoc</code> now also requires the user to have an email (login, refresh and org switch).	For email-less users, <code>null === null</code> would wrongly mark them as an organisation's point of contact. Users with an email get the same result as before.
<code>src/middleware/jwt.middleware.ts</code>	(a) <code>/auth/student/signup</code> and <code>/auth/student/login</code> added to the public routes. (b) A token with no email now matches the user on <code>email IS NULL</code> .	(a) Lets students sign up and log in without a token. (b) The old check <code>email = <token email></code> can never match NULL in SQL, so every request from these students would have returned 401. Tokens that carry an email run the exact same SQL as before.
<code>src/controller/classes/classes.service.ts</code>	Students without an email are left out of Google Calendar and Zoom invite lists (class creation and session merge).	A <code>null</code> attendee makes Google reject the whole event, which would have broken class creation for the entire batch.
<code>src/services/attendance-worker/attendance-worker.service.ts</code>	Zoom attendance matching ignores empty emails.	Zoom reports guests with an empty email; without this, a guest could be credited to a student who has no email.
<code>src/services/attendance/attendance-calculation.service.ts</code>	Google Meet attendance matching ignores empty emails.	Same reason as above.
<code>src/controller/student/student.service.ts</code>	<code>email.toLowerCase()</code> → <code>email?.toLowerCase() ?? null</code> in "completed classes with attendance".	It would have crashed for a student without an email.
<code>drizzle/schema.ts</code>	Added the <code>zuvyStudentCredentials</code> table and its relation.	Drizzle definition of the new table.
<code>src/app.module.ts</code>	Registered <code>StudentAuthModule</code> .	Turns on the new endpoints.
<code>src/main.ts</code>	<code>app.set('trust proxy', ...)</code> , trusting only nginx's addresses (loopback and Docker's private ranges).	Without it, every request looks like it comes from nginx, so a per-IP limit would be one limit for everyone. No existing code reads the client IP, protocol or hostname, so other routes behave exactly as before.
<code>sample.env</code>	Added the new optional settings (section 3.2).	Documentation for deployments.

3. Before you test: migration and server settings

3.1 Run the migration

The new endpoints need the new table. **Run the migration on the dev database first** (and later on stage and production, using your normal process).

1. Connect to the dev database with `psql`, DBeaver or pgAdmin.
2. Select the right schema. It is `main` unless the server runs with `ENV_NOTE=stage_template`:

```
SET search_path TO main;           -- or: SET search_path TO stage_template;
```

3. Run the whole file `drizzle/migrations/0043_create_zuvy_student_credentials.sql`.
4. Check that the table exists:

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'zuvy_student_credentials'
ORDER BY ordinal_position;
```

You should see 10 columns.

Deploying the code **before** the migration is safe for existing users, because nothing else reads this table. Only the new `/auth/student/*` endpoints fail, with a 500 error, until the table exists.

3.2 Environment variables (`.env` on the EC2 server)

Variable	Required	Example / default	Meaning
<code>TURNSTILE_SECRET_KEY</code>	Yes (unless the captcha is disabled on dev)	From the Cloudflare dashboard	Secret key used to verify captcha tokens. The frontend uses the matching site key .
<code>TURNSTILE_ALLOWED_HOSTNAMES</code>	No	<code>app.zuvy.org, dev.zuvy.org</code>	If set, the captcha must have been solved on one of these sites. Leave empty when using Cloudflare test keys.
<code>CAPTCHA_DISABLED</code>	No	<code>true</code> on dev only	Skips the captcha check so testers can use Swagger. Ignored on production (when <code>BASE_URL</code> contains <code>main-api</code>).
<code>SIGNUP_LIMIT_PER_MINUTE</code>	No	<code>60</code>	Sign-ups allowed per IP per minute
<code>SIGNUP_LIMIT_PER_HOUR</code>	No	<code>300</code>	Sign-ups allowed per IP per hour
<code>LOGIN_LIMIT_PER_15_MINUTES</code>	No	<code>600</code>	Student ID login attempts allowed per IP per 15 minutes
<code>TRUST_PROXY</code>	No	<code>loopback, uniquelocal</code>	Proxies trusted for the client IP. The default fits nginx on the same EC2 host in front of Docker.

Cloudflare test keys for dev (they never show a real puzzle):

Purpose	Value
Test secret, always passes	1x000000000000000000000000000000AA
Test secret, always fails	2x000000000000000000000000000000AA
Test site key for the frontend, always passes	1x000000000000000000000000AA
Token to send with test keys	XXXX.DUMMY.TOKEN.XXXX

Restart the container after changing `.env` (`docker compose -f compose.yaml up -d --force-recreate`).

3.3 nginx and Docker: make the real client IP reach the API

The API runs in Docker behind nginx on EC2. For the per-IP limits to see each student's own IP, and not nginx's, check these two things on the server once.

1. **nginx forwards the client IP.** In the `location` block that proxies to the API:

```
location / {
    proxy_pass http://127.0.0.1:5001;    # your existing upstream
    proxy_set_header Host                $host;
    proxy_set_header X-Real-IP           $remote_addr;
    proxy_set_header X-Forwarded-For     $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Then run `sudo nginx -t && sudo systemctl reload nginx`.

2. **The container port is only reachable through nginx.** `compose.yaml` publishes `"5001:5000"` on every network interface, so someone could skip nginx and send a fake `X-Forwarded-For` header. If nginx's `proxy_pass` points to `127.0.0.1` or `localhost`, bind the port to localhost:

```
ports:
  - "127.0.0.1:5001:5000"
```

Check first with `grep -R proxy_pass /etc/nginx/`. If nginx reaches the API on another address, keep the port as it is and block port 5001 from the internet in the EC2 security group instead.

If Cloudflare or another CDN sits in front of nginx, nginx also needs its `real_ip` module set up with that provider's IP ranges. Otherwise every request will carry the CDN's IP.

4. Using Swagger

1. Open the dev Swagger page: `<dev-api-url>/apis` (locally: `http://localhost:<PORT>/apis`). Swagger is turned off on production.
2. Find the section **"Student ID Authentication"**. All 5 new endpoints are there.
3. **Calling an endpoint:** expand it, click **Try it out**, edit the request body, then click **Execute**.
4. **Calling a protected endpoint:**
 1. Copy the `access_token` from a sign-up or login response.

2. Click the **Authorize** button (padlock, top right of the page).
3. Paste **only the token**, without the word `Bearer`. Swagger adds `Bearer` itself.
4. Click **Authorize**, then **Close**.
5. **Switching user** (for example from a student to a super admin): click **Authorize** → **Logout**, then authorize again with the other token.

The captcha in Swagger

Swagger can't show the captcha widget, so on the dev server use one of these:

Dev <code>.env</code> setting	What to send as <code>captchaToken</code> in sign-up
<code>CAPTCHA_DISABLED=true</code>	Leave <code>captchaToken</code> out (or send anything)
<code>TURNSTILE_SECRET_KEY=1x00000000000000000000000000000000AA</code> (Cloudflare test key)	<code>"XXXX.DUMMY.TOKEN.XXXX"</code> . This runs the real verification code against Cloudflare.

On production neither works: a real token from the frontend widget is always required.

Getting a super admin token (needed for the admin APIs)

The admin endpoints (`/auth/student/admin/*`) only work for a **super admin**.

1. Log in to the **dev Zuvy web app** with a Google account that is a super admin on dev.
2. Open the browser DevTools (F12) → **Network** tab.
3. Click any request that goes to the API and copy the value after `Authorization: Bearer` in the request headers.
4. Use that token in Swagger's **Authorize** dialog.

If you don't have a super admin account on dev, ask an existing super admin to add your email through `POST /super-admin`.

5. API reference with examples

All examples use `ZV7K3M9Q` as the Student ID. Use the ID you actually receive.

5.1 Sign up: `POST /auth/student/signup`

Creates the account and logs the student in straight away.

- **Login needed:** no (public)
- **Request body:**

```
{
  "name": "Riya Sharma",
  "password": "riya@2026",
  "captchaToken": "XXXX.DUMMY.TOKEN.XXXX"
}
```

- **Rules:**

- `name` : 2–100 characters. Spaces at the start and end are removed, and repeated spaces are collapsed to one.
- `password` : 6–64 characters.
- `captchaToken` : the token from the Turnstile widget. Required unless `CAPTCHA_DISABLED=true` on dev (see section 4).
- No other fields are allowed. For example, sending `email` returns 400.
- **Order of checks:** per-IP limit → input validation → captcha → account creation. A failed captcha never creates anything in the database.
- **Success:** `201 Created`

```
{
  "studentId": "ZV7K3M9Q",
  "access_token": "eyJhbGciOi...",
  "refresh_token": "eyJhbGciOi...",
  "showTooltip": true,
  "user": {
    "id": "48213",
    "email": null,
    "name": "Riya Sharma",
    "profilePicture": null,
    "role": "student",
    "center": null,
    "rolesList": ["student"],
    "orgId": null,
    "orgName": null,
    "isPoc": false,
    "permissions": {}
  }
}
```

Everything except `studentId` has exactly the same shape as the Google login response (`POST /auth/login`).

- **Errors:**

Status	When	Example message
400	Name too short or long, password too short or long, or an extra field	<code>["password must be longer than or equal to 6 characters"]</code>
400	No captcha token	<code>Captcha is required</code>
400	Cloudflare rejected the token (invalid, expired, already used, or solved on another site)	<code>Captcha verification failed. Please try again.</code>
429	Too many sign-ups from this IP (section 1)	<code>Too many requests from this network. Please wait a few minutes and try again.</code>
503	<code>TURNSTILE_SECRET_KEY</code> not set on the server	<code>Sign-up is temporarily unavailable. Please try again later.</code>
503	Cloudflare couldn't be reached	<code>Could not verify the captcha. Please try again.</code>

Status	When	Example message
500	Table missing (migration not run) or a database error	Could not create the account

5.2 Log in: `POST /auth/student/login`

- **Login needed:** no (public)
- **Request body:**

```
{
  "studentId": "ZV7K3M9Q",
  "password": "riya@2026"
}
```

`studentId` is case-insensitive and spaces around it are ignored.

- **Success: 200 OK:** same shape as sign-up (`studentId` , `access_token` , `refresh_token` , `showTooltip` , `user`). `showTooltip` is `false` after the first login.
- **Errors:**

Status	When	Message
400	Field missing or empty	validation message
401	Wrong Student ID or wrong password (deliberately the same message)	Invalid Student ID or password
429	5 wrong passwords in a row; this account is locked for 15 minutes	Too many wrong attempts. Try again after 15 minute(s).
429	Too many login attempts from this IP (default 600 per 15 minutes)	Too many requests from this network. Please wait a few minutes and try again.

Login has no captcha. Young students log in often, and the account lockout plus the per-IP ceiling already stop password guessing.

5.3 Reset a password (super admin): `POST /auth/student/admin/reset-password`

Use this when a student forgets their password.

- **Login needed:** yes, as a super admin (Authorize in Swagger first)
- **Request body:** set a specific password

```
{
  "studentId": "ZV7K3M9Q",
  "newPassword": "newpass123"
}
```

- **Request body:** let the server generate one (8 easy-to-read characters)

```
{
  "studentId": "ZV7K3M9Q"
}
```

- Success: 200 OK

```
{
  "status": "success",
  "message": "Password updated. Share it with the student; it is not shown again.",
  "data": {
    "studentId": "ZV7K3M9Q",
    "userId": "48213",
    "name": "Riya Sharma",
    "password": "k7mq4x2d"
  }
}
```

The reset also **removes any logout**. Who reset it and when is saved in `last_reset_by` and `last_reset_at`.

- Errors:

Status	When	Message
400	<code>newPassword</code> shorter than 6 or longer than 64 characters	validation message
401	No token, or an invalid token	<code>Token not found</code> / <code>Invalid token</code>
403	Caller is not a super admin	<code>Only super admins can manage Student IDs</code>
404	Student ID does not exist	<code>Student ID not found</code>

5.4 Find students (super admin): `GET /auth/student/admin/students`

Use this to look up a student's Student ID by name, or to check an account.

- Login needed: yes, as a super admin
- Query parameters (pass `studentId` or `name`, or both):

Param	Example	Meaning
<code>studentId</code>	<code>ZV7K3M9Q</code>	Exact Student ID
<code>name</code>	<code>riya</code>	Part of the name (case-insensitive)
<code>limit</code>	<code>20</code>	1–100, default 20

- Example: `GET /auth/student/admin/students?name=riya`
- Success: 200 OK

```
{
  "status": "success",
  "data": [
    {
      "userId": "48213",
      "studentId": "ZV7K3M9Q",
      "name": "Riya Sharma",
      "createdAt": "2026-10-08 10:15:22.481+00",
      "lastLoginAt": "2026-10-08 10:20:03.112+00",
      "lockedUntil": null,
      "lastResetAt": null
    }
  ]
}
```

- **Errors:** 400 (neither `studentId` nor `name` given: Pass a studentId or a name to search), 401, 403.

5.5 Enrol in a course (super admin): `POST /auth/student/admin/enroll`

The existing admin "add students" API (`POST /bootcamp/students/:bootcamp_id`) finds students **by email**, so it can't add these students. Use this endpoint for **private** courses instead. Public courses don't need it.

- **Login needed:** yes, as a **super admin**
- **Request body:**

```
{
  "studentId": "ZV7K3M9Q",
  "bootcampId": 12,
  "batchId": 34
}
```

`batchId` is optional, but **always send it**. `GET /student/` (the student's "My courses") only lists enrolments that have a batch. Without `batchId`, the student is enrolled but won't see the course until an admin assigns them to a batch.

- **Success:** `200 OK`

```
{ "status": "success", "message": "Student ZV7K3M9Q enrolled in Full Stack Web Development" }
```

If the student is already in the course and you pass a **different** `batchId`:

```
{ "status": "success", "message": "Student moved to the selected batch" }
```

- **Errors:**

Status	When	Message
400	Batch is full	The maximum capacity for the batch has been reached
403	Caller is not a super admin	Only super admins can manage Student IDs

Status	When	Message
404	Wrong Student ID, course or batch	Student ID not found / Course not found / Batch not found in this course
409	Already enrolled (same batch, or no batch given)	Student is already enrolled in this course

5.6 Existing endpoints that work unchanged for these students

Endpoint	Body or note
POST /auth/refresh	{ "refresh_token": "<refresh_token>" }; returns a new pair, and the old refresh token stops working
POST /auth/logout	Authorize with the student's access token; afterwards that token returns 401
GET /student/	Courses the student is enrolled in
GET /student/bootcamp/public	Public courses
GET /tracking/allModulesForStudents/{bootcampId}	Modules of a course (also auto-enrols in a public course)
GET /tracking/bootcampProgress/{bootcampId}	Course progress
GET /student/leaderboard/{bootcampId}	Leaderboard (the student appears with email: null)

6. Test cases: step by step

Write down the `studentId`, `access_token` and `refresh_token` from TC-01; later tests use them.

Sections A–E assume the dev server has `CAPTCHA_DISABLED=true`, so sign-up bodies can leave out `captchaToken`. If dev uses the Cloudflare test secret instead, add `"captchaToken": "XXXX.DUMMY.TOKEN.XXXX"` to every sign-up body. Section F tests the captcha and the rate limits themselves.

A. Sign-up and login

ID	Steps	Expected result
TC-01	POST /auth/student/signup with {"name": "Test Student One", "password": "test@123"}	201. The response has a <code>studentId</code> starting with <code>ZV</code> and 8 characters long, plus <code>access_token</code> , <code>refresh_token</code> , <code>user</code> . <code>email = null</code> and <code>user.rolesList = ["student"]</code> .
TC-02	Sign up again with the same name and password	201 with a different <code>studentId</code> . Names don't have to be unique.
TC-03	Sign up with {"name": "A", "password": "test@123"}	400: the name is too short.
TC-04	Sign up with {"name": "Test", "password": "123"}	400: the password is too short.

ID	Steps	Expected result
TC-05	Sign up with <code>{"name": "Test", "password": "test@123", "email": "a@b.com"}</code>	400: <code>property email should not exist</code> .
TC-06	<code>POST /auth/student/login</code> with the Student ID from TC-01 and <code>test@123</code>	200. Same shape as TC-01, and <code>showTooltip</code> is <code>false</code> .
TC-07	Log in with the Student ID in lowercase	200.
TC-08	Log in with the right ID and a wrong password	401 <code>Invalid Student ID or password</code> .
TC-09	Log in with an ID that doesn't exist (e.g. <code>ZVAAAAAA</code>)	401 with the same message as TC-08.

B. Lockout

ID	Steps	Expected result
TC-10	Use the TC-02 account. Log in with a wrong password 5 times.	Attempts 1–5 return 401.
TC-11	Right after that, log in with the correct password	429 <code>Too many wrong attempts. Try again after 15 minute(s)</code> .
TC-12	Reset the password as a super admin (TC-20), then log in with the new password	200: the reset removed the lock.

C. Using the token on normal student APIs

Authorize in Swagger with the student's `access_token` first.

ID	Steps	Expected result
TC-13	<code>GET /student/</code>	200, with <code>inProgressBootcamps</code> and <code>completedBootcamps</code> both empty for a new student. There must be no 401: this proves the middleware fix works.
TC-14	<code>GET /student/bootcamp/public</code> and pick a public course id, then call <code>GET /tracking/allModulesForStudents/{id}</code>	200 with the modules.
TC-15	<code>GET /student/</code> again	The public course from TC-14 now appears in <code>inProgressBootcamps</code> (automatic enrolment).
TC-16	Open a chapter, mark it complete and submit a quiz or coding problem through the normal student APIs or the web app	Works the same as for a Google student. Progress and leaderboard update.
TC-17	<code>POST /auth/refresh</code> with <code>{"refresh_token": "<refresh_token>"}</code>	200 with a new <code>access_token</code> and <code>refresh_token</code> .
TC-18	Call <code>POST /auth/refresh</code> again with the old refresh token	401.
TC-19	<code>POST /auth/logout</code> (authorized as the student), then <code>GET /student/</code> with the same token	Logout returns 201; the next call returns 401.

D. Super admin tools

Authorize in Swagger with a **super admin** token first.

ID	Steps	Expected result
TC-20	POST /auth/student/admin/reset-password with {"studentId": "<id from TC-02>"}	200 with a generated 8-character password .
TC-21	Log in as that student with the old password	401.
TC-22	Log in with the password from TC-20	200.
TC-23	Reset with {"studentId": "<id>", "newPassword": "mychoice1"}, then log in with mychoice1	Both 200.
TC-24	Reset with {"studentId": "ZVAAAAAA"}	404 Student ID not found .
TC-25	GET /auth/student/admin/students?name=test student	200; both test accounts are listed.
TC-26	GET /auth/student/admin/students with no parameters	400.
TC-27	POST /auth/student/admin/enroll with a private course: {"studentId": "<id>", "bootcampId": <id>, "batchId": <id>}	200 "enrolled in ...". As the student, GET /student/ now lists the course.
TC-28	Repeat TC-27 exactly	409 Student is already enrolled in this course .
TC-29	Repeat TC-27 with another batch of the same course	200 Student moved to the selected batch .
TC-30	Enrol with a batchId from a different course	404 Batch not found in this course .
TC-31	Authorize as a normal student, admin or instructor (not a super admin) and call any /auth/student/admin/* endpoint	403 Only super admins can manage Student IDs .
TC-32	Log out of Swagger (no token) and call any /auth/student/admin/* endpoint	401.

E. Classes and attendance (needs a batch with a Student ID student in it)

ID	Steps	Expected result
TC-33	As an admin or instructor, create a Google Meet class for a batch that contains a Student ID student and email students	Class is created. Email students get the calendar invite.
TC-34	Create a Zoom class for the same batch	Class is created. Email students are invited.
TC-35	After the Zoom class ends and attendance is processed	Email students' attendance is recorded as before. The Student ID student is not matched automatically (see section 9).

ID	Steps	Expected result
TC-36	As an admin, mark the Student ID student present with <code>POST /bootcamp/attendance/mark</code> , body <code>[{"sessionId": <id>, "userId": <student user id>, "status": "present"}]</code> . Send this flat list, not the <code>{"attendance": [...]}</code> wrapper that Swagger suggests: the server reads <code>sessionId</code> from each item directly.	Attendance is saved for that student.

F. Captcha and rate limits (needs access to the dev server's `.env`)

Change `.env`, restart the container (section 3.2), and run the test. Put the original values back afterwards.

ID	<code>.env</code> on dev	Steps	Expected result
TC-37	<code>TURNSTILE_SECRET_KEY=1x0000000000000000000000000000AA</code> , no <code>CAPTCHA_DISABLED</code>	Sign up without <code>captchaToken</code>	400 <code>Captcha is required</code>
TC-38	Same as TC-37	Sign up with <code>"captchaToken": "XXXX.DUMMY.TOKEN.XXXX"</code>	201: the real Cloudflare check ran and passed
TC-39	<code>TURNSTILE_SECRET_KEY=2x0000000000000000000000000000AA</code> (always fails)	Sign up with the dummy token	400 <code>Captcha verification failed. Please try again.</code> No new row in <code>zuvy_student_credentials</code> (section 8).
TC-40	No <code>TURNSTILE_SECRET_KEY</code> and no <code>CAPTCHA_DISABLED</code>	Sign up	503 <code>Sign-up is temporarily unavailable. Please try again later.</code>
TC-41	<code>CAPTCHA_DISABLED=true</code> , <code>SIGNUP_LIMIT_PER_MINUTE=3</code>	Sign up 4 times within one minute from the same network (e.g. office Wi-Fi)	Calls 1–3 return 201; call 4 returns 429 <code>Too many requests from this network...</code> . The response has a <code>retry-after-signupPerMinute</code> header (seconds to wait). Allowed calls carry <code>x-ratelimit-remaining-signupPerMinute</code> .
TC-42	Same as TC-41	While the office network is still blocked, sign up from a different network (e.g. a phone hotspot)	201. <i>This proves nginx passes the real client IP.</i> If it's also 429, every user is sharing nginx's IP: fix section 3.3.
TC-43	Same as TC-41	From the blocked network, try to fake a new IP with curl (example below)	Still 429: a client's own <code>X-Forwarded-For</code> value is ignored.
TC-44	<code>LOGIN_LIMIT_PER_15_MINUTES=3</code>	Log in 4 times from the same network	Calls 1–3 return 200 (or 401 for a wrong password); call 4 returns 429. A different network can still log in.

ID	.env on dev	Steps	Expected result
TC-45	Any of the low limits above	Call normal APIs (e.g. <code>GET /student/</code> , Google login, admin pages) many times	Never 429 : only the two Student ID routes are rate limited.
TC-46	Original values restored	Sign up and log in normally	Works as in sections A–E.

Curl for TC-43 (replace the URL):

```
curl -i -X POST "https://<dev-api-url>/auth/student/signup" \
-H "Content-Type: application/json" \
-H "X-Forwarded-For: 1.2.3.4" \
-d '{"name": "Spoof Test", "password": "test@123"}'
```

7. Regression tests for users with email

These confirm that nothing changed for existing users. **Run them before releasing to production.**

ID	Steps	Expected result
RG-01	Log in to the web app with Google as a student	Works. Courses, progress and leaderboard look normal.
RG-02	Log in with Google as an instructor	Works. Instructor dashboard and classes load.
RG-03	Log in with Google as an admin	Works. Courses, batches and students load.
RG-04	Log in with Google as a super admin	Works.
RG-05	Check the login response (Network tab) for an org POC user	<code>isPoc</code> is <code>true</code> for the POC and <code>false</code> for everyone else, the same as before.
RG-06	As an admin in 2 organisations, switch org (<code>POST /org/switch-org</code>)	Works and returns new tokens.
RG-07	Stay idle until the token refreshes, or call <code>POST /auth/refresh</code>	Works.
RG-08	Log out, then try the old token	401, as before.
RG-09	Create a Google Meet class and a Zoom class for a normal batch	Both are created, and invites are sent to all students.
RG-10	After a Zoom class, check attendance	Recorded for students who attended, as before.
RG-11	Merge two sessions (<code>POST /classes/merge</code>)	Works; child-batch students are added to the parent session.
RG-12	Add students by email to a course (<code>POST /bootcamp/students/{id}</code>)	Works as before.

8. Checking the database

Run `SET search_path TO main;` (or `stage_template`) first.

Latest Student ID accounts:

```
SELECT c.student_id, u.id AS user_id, u.name, u.email, u.google_user_id, u.mode,
       c.failed_attempts, c.locked_until, c.last_reset_by, c.last_reset_at, c.created_at
FROM zuvy_student_credentials c
JOIN users u ON u.id = c.user_id
ORDER BY c.created_at DESC
LIMIT 10;
```

Expected: `email` and `google_user_id` are NULL, and `mode` is `student`.

Passwords are never stored in plain text:

```
SELECT student_id, LEFT(password_hash, 20) AS hash_start
FROM zuvy_student_credentials
ORDER BY created_at DESC
LIMIT 5;
```

Expected: every value starts with `scrypt$16384$8$1$`.

Enrolments of a Student ID account:

```
SELECT e.bootcamp_id, e.batch_id, e.status, e.enrolled_date
FROM zuvy_batch_enrollments e
JOIN zuvy_student_credentials c ON c.user_id = e.user_id
WHERE c.student_id = 'ZV7K3M9Q';
```

Unlock an account by hand (the reset API does the same):

```
UPDATE zuvy_student_credentials
SET failed_attempts = 0, locked_until = NULL
WHERE student_id = 'ZV7K3M9Q';
```

Delete test accounts (deleting the user also deletes the credential row):

```
DELETE FROM users
WHERE id IN (
  SELECT user_id FROM zuvy_student_credentials
  WHERE student_id IN ('ZV7K3M9Q')
);
```

If the delete fails because of rows in other tables (enrolments, submissions and so on), remove those test rows first, or just leave the test accounts on dev.

9. Known limitations

Limitation	Workaround or plan
Live-class attendance is not recorded automatically for Student ID students. Zoom and Google Meet identify people by email, and these students join as guests.	An admin marks them with <code>POST /bootcamp/attendance/mark</code> (TC-36).
Students without an email get no calendar or Zoom invites .	They join from the class link in the Zuvy app.
Old sessions stay valid after a password reset, until the tokens expire (up to 7 days through refresh). Student tokens are not stored on the server, so they can't be cancelled.	Acceptable for forgotten passwords.
Rate-limit counters are kept in the container's memory . They reset when the container restarts, and if the API is scaled to several containers, each one counts separately.	Fine for one container. For several, switch the throttler to Redis storage.
A very large school (more than about 300 sign-ups an hour from one public IP) could reach the sign-up ceiling.	Raise <code>SIGNUP_LIMIT_PER_HOUR</code> / <code>SIGNUP_LIMIT_PER_MINUTE</code> in <code>.env</code> . No code change is needed.
Sign-up depends on Cloudflare. If Cloudflare is unreachable, sign-up returns 503. Login is not affected.	Rare. Students can retry a few minutes later.
A determined person can still solve captchas by hand and create accounts slowly.	Watch for spikes. Class join codes with seat limits would remove this completely (future work).
These accounts must stay students . Giving them an admin or instructor role breaks their login, because staff sessions are stored with an email column that can't be empty.	Don't assign roles to Student ID accounts.
Only super admins can reset passwords, search students and enrol them in private courses.	Org admins ask a super admin.
There is no parental consent step, which the DPDP Act requires for under-18 learners.	Product or legal to decide (for example a consent checkbox on sign-up).

10. Troubleshooting

You see	Likely cause	Fix
Sign-up returns 500 <code>Could not create the account</code>	The migration hasn't been run on this database	Run section 3
Every student API returns 401 with a Student ID token	The server is running old code without the middleware change	Deploy the latest build
<code>/auth/student/signup</code> returns 401 <code>Token not found</code>	Old middleware without the public routes	Deploy the latest build
Admin APIs return 403	The token isn't a super admin's	Use a super admin token (section 4)

You see	Likely cause	Fix
429 on login Too many wrong attempts...	The account is locked after 5 wrong passwords	Wait 15 minutes, or reset the password
429 Too many requests from this network...	The per-IP limit was reached	Wait the number of seconds in the <code>retry-after-<limit name></code> header, or raise the limit in <code>.env</code>
Everyone gets 429 at the same time, from different places	nginx isn't forwarding the client IP, so all users share nginx's IP	Fix the nginx headers and check <code>TRUST_PROXY</code> (section 3.3)
Sign-up 400 Captcha is required	<code>captchaToken</code> is missing	Send the token, or set <code>CAPTCHA_DISABLED=true</code> on dev
Sign-up 400 Captcha verification failed	The token is invalid, expired (older than 5 minutes), already used, or from another site	Get a fresh token from the widget. With test keys, leave <code>TURNSTILE_ALLOWED_HOSTNAMES</code> empty.
Sign-up 503 Sign-up is temporarily unavailable	<code>TURNSTILE_SECRET_KEY</code> isn't set on the server	Add it to <code>.env</code> and restart the container
400 property X should not exist	The body has an extra field	Send only the fields shown in section 5
Endpoints missing in Swagger	Old build, or you're on production (Swagger is off there)	Use the dev Swagger page after deploying

11. Notes for frontend developers

- **New login option.** Add a "Login with Student ID" option next to Google: call `POST /auth/student/login` with `{ studentId, password }`.
- **New sign-up page.** It takes name and password and calls `POST /auth/student/signup`.
- **Captcha widget on sign-up.** Add the Cloudflare Turnstile widget with the **site key** (dev: test key `1x00000000000000000000AA`). Send its token as `captchaToken` in the sign-up body. A token works **once** and expires after 5 minutes, so reset the widget after any failed sign-up to get a new token.
- **Handle the response like Google login.** It has the same shape, plus `studentId`: store `access_token` and `refresh_token` the same way and use the same refresh and logout calls.
- **Show the Student ID prominently after sign-up.** For example, a large card with a "Download / print" option. It is the student's only way to log in, and there's no "forgot password".
- **Handle a missing email.** `user.email` is `null` for these students; anywhere the UI shows the user's email should handle that, for example by showing the Student ID instead.
- **Map error codes to messages:**
 - 401: "Wrong Student ID or password"
 - 429: show the server message (it says how many minutes to wait)
 - 400: show the validation or captcha message, and reset the captcha widget
 - 503 on sign-up: "Sign-up is temporarily unavailable, please try again later"